

1. Why do lambdas require Functional Interfaces?

Answer:

A lambda expression doesn't have a type of its own. It needs a **target type**, and that target type is a **Functional Interface** (an interface with exactly one abstract method). Java uses the Functional Interface to determine which method the lambda is implementing.

Example

```
@FunctionalInterface
interface Greeting {
    void greet();
}

Greeting g = () -> System.out.println("Hello");
```

Here, the lambda implements greet().

One-liner:

A lambda requires a Functional Interface because it needs a target type with exactly one abstract method.

2. Difference between Predicate and Function?

Predicate	Function
Checks a condition	Transforms a value
Returns boolean	Returns any type R
Method: test()	Method: apply()

Example

Predicate

```
Predicate isEven = n -> n % 2 == 0;
```

```
System.out.println(isEven.test(8));
```

Output

```
true
```

Function

```
Function square = n -> n * n;  
System.out.println(square.apply(8));
```

Output

```
64
```

Interview Answer

Predicate is used for condition checking and always returns boolean, whereas Function transforms one value into another and returns the transformed value.

3. Which interface returns void?

Answer

Consumer

Method

```
void accept(T t)
```

Example

```
Consumer print = System.out::println;  
print.accept("Java");
```

Output

```
Java
```

Why?

Because a Consumer only performs an action and doesn't return anything.

One-liner

Consumer consumes a value and performs an action without returning anything.

4. Which interface takes no parameters?

Answer

Supplier

Method

```
T get();
```

Example

```
Supplier random = Math::random;  
System.out.println(random.get());
```

Output (example)

```
0.726541
```

Why?

A Supplier's job is to **supply** a value whenever someone asks for it.

One-liner

Supplier takes no input and returns a value using the `get()` method.

5. Why is Supplier called lazy?

Answer

Because it **doesn't create or compute the value immediately**. It waits until `get()` is called.

Example

```
Supplier> supplier = ArrayList::new;

System.out.println("Nothing created yet.");

List list = supplier.get();

System.out.println("List created.");
```

The `ArrayList` object is created **only** when `get()` executes.

Real Project Example

```
Optional optional = Optional.empty();

String value = optional.orElseGet(() -> "Default Value");
```

"Default Value" is created **only if** the `Optional` is empty.

Interview One-liner

Supplier is called lazy because it creates or provides a value only when `get()` is invoked.

6. Difference between a lambda and a method reference?

A **method reference** is simply a shorter form of a lambda **when the lambda only calls an existing method**.

Lambda

```
Consumer print = s -> System.out.println(s);
```

Method Reference

```
Consumer print = System.out::println;
```

Both do exactly the same thing.

Another Example

Lambda

```
Function upper =  
    s -> s.toUpperCase();
```

Method Reference

```
Function upper =  
    String::toUpperCase;
```

Interview Answer

A method reference is a shorthand for a lambda that simply invokes an existing method.

7. Can every lambda be replaced with a method reference?

Answer

No.

Only when the lambda **directly calls an existing method without adding extra logic**.

Can be replaced

```
Consumer print =  
    s -> System.out.println(s);
```

↓

```
Consumer print =  
    System.out::println;
```

Cannot be replaced

```
Function square =  
    x -> {  
        System.out.println(x);  
        return x * x;  
    };
```

This lambda performs **two operations**:

- Prints
- Squares

There is no single existing method that does both.

Interview One-liner

Only simple lambdas that directly call an existing method can be replaced by a method reference.

8. Which Stream methods commonly use Predicate, Function, and Consumer?

Predicate → `filter()`

```
List nums = List.of(1,2,3,4,5,6);

nums.stream()
    .filter(n -> n % 2 == 0)
    .forEach(System.out::println);
```

Output

```
2
4
6
```

Function → `map()`

```
List names = List.of("java","spring");

names.stream()
    .map(String::toUpperCase)
    .forEach(System.out::println);
```

Output

```
JAVA
SPRING
```

Consumer → `forEach()`

```
List names = List.of("Ali","Ahmed");  
  
names.forEach(System.out::println);
```

Output

```
Ali  
Ahmed
```

Interview Answer

- filter() → Predicate
 - map() → Function
 - forEach() → Consumer
-

9. Which built-in interface would you use for validation?

Answer

Predicate

Validation is simply checking whether something satisfies a condition.

Example

```
Predicate validPassword =  
    password -> password.length() >= 8;  
  
System.out.println(validPassword.test("Java1234"));
```

Output

```
true
```

Real-world Example


```
Predicate isAdult =  
    user -> user.getAge() >= 18;
```

Interview One-liner

Use Predicate whenever you need validation or condition checking.

10. Which built-in interface would you use to convert an object into another type?

Answer

Function

Example

Convert String → Integer

```
Function parser =  
    Integer::parseInt;  
  
System.out.println(parser.apply("100"));
```

Output

```
100
```

Another Example

Convert Employee → Name

```
Function getName =  
    Employee::getName;
```

Used with Streams

```
employees.stream()  
    .map(Employee::getName)  
    .toList();
```

Interview One-liner

Use Function whenever you need to transform one object into another.

★ Interview Rapid Fire (30 Seconds Revision)

Question	Short Answer
Why do lambdas need Functional Interfaces?	They need a target type with one abstract method.
Which interface checks conditions?	Predicate
Which interface transforms data?	Function
Which interface returns nothing?	Consumer
Which interface takes no input?	Supplier
Why is Supplier lazy?	It creates/provides values only when get() is called.
Lambda vs Method Reference?	Method reference is shorthand for a lambda that calls an existing method.
Can every lambda become a method reference?	No. Only simple method-calling lambdas.
Which Stream method uses Predicate?	filter()
Which Stream method uses Function?	map()
Which Stream method uses Consumer?	forEach()
Which interface is used for validation?	Predicate

Which interface is used for object conversion?	Function
Predicate method name?	test()
Function method name?	apply()
Consumer method name?	accept()
Supplier method name?	get()